

Dynamic Programming Templates

Six template families. Every problem maps to one loop skeleton and one dp-state definition.

Quick Reference Table

#	Name	Category	dp state	Init	Key transition
10	Regular Expression Matching	Match string s against pattern p with . and *	2D DP: $dp[i][j]$ = s[0..i] matches p[0..j]; '*' means zero or more of preceding char	*** two cases: zero occurrence $dp[i][j-2]$, or match $dp[i-1][j]$ if chars match	$O(m \cdot n)$
70	Climbing Stairs	Linear 1D	$dp[i]$ = ways to reach step i	$dp[1]=1$, $dp[2]=2$	$dp[i-1]+dp[i-2]$
19 8	House Robber	Linear 1D	$dp[i]$ = max rob up to i	$dp[0]=nums[0]$	$\max(dp[i-1], dp[i-2]+nums[i])$
21 3	House Robber II	Linear 1D	same, two runs	skip first or skip last	max of two runs
30 0	LIS	Look-back 1D	$dp[i]$ = LIS ending at i	$dp[i]=1$	$\max(dp[j]+1)$ if $nums[j] < nums[i]$
13 9	Word Break	Look-back 1D	$dp[i]$ = can break s[0..i)	$dp[0]=true$	$dp[j]$ && word[j..i) in dict
41 6	Partition Equal Subset	Knapsack 0/1	$dp[j]$ = can make sum j	$dp[0]=true$	$dp[j]] = dp[j-num]$
49 4	Target Sum	Knapsack 0/1	$dp[j]$ = ways to reach sum j	$dp[0]=1$	$dp[j] += dp[j-num]$
47 4	Ones and Zeroes	Knapsack 0/1 2D	$dp[i][j]$ = max strings with i zeros j ones	0	$\max(dp[i][j], dp[i-z][j-o]+1)$
32 2	Coin Change	Knapsack Unbounded	$dp[j]$ = min coins for amount j	$dp[0]=0$, rest=INF	$\min(dp[j], dp[j-coin]+1)$
51 8	Coin Change II	Knapsack Unbounded	$dp[j]$ = ways to make amount j	$dp[0]=1$	$dp[j] += dp[j-coin]$
58 3	Delete Operation for Two Strings	2D Sequence (LCS)	$dp[i][j]$ = LCS of word1[..i) and word2[..j)	$dp[0][*]=0$	match: +1, else $\max(up, left)$
37 7	Combination Sum IV	Permutation count	$dp[j]$ = ordered arrangements for j	$dp[0]=1$	target outer, nums inner
11 43	LCS	2D Sequence	$dp[i][j]$ = LCS of s1[..i) and s2[..j)	$dp[0][*]=0$	match: +1, else $\max(up, left)$
72	Edit Distance	2D Sequence	$dp[i][j]$ = edits to convert s1[..i) to s2[..j)	$dp[i][0]=i$	match: diag, else $1+\min 3$
87	Scramble String	Is s2 a scramble of s1 via recursive partition swaps?	3D DP/memo: $dp[i][j][len]$; try every split point, with and without swap	Split + optional swap: check both swapped and non-swapped partitions	$O(n^4)$
91	Decode Ways	Look-back 1D	$dp[i]$ = ways to decode s[0..i)	$dp[0]=1$, $dp[1]=1$	$dp[i] += dp[i-1]$ if 1-digit valid; $dp[i] += dp[i-2]$ if 2-digit valid
96	Unique Binary Search Trees	1D DP (Catalan)	$dp[i]$ = count of unique BSTs with i nodes	$dp[0]=dp[1]=1$	$dp[i] += dp[j-1]*dp[i-j]$ for $j=1..i$
11 5	Distinct Subsequences	2D Sequence	$dp[i][j]$ = ways s[..i) contains t[..j)	$dp[i][0]=1$	skip+match
64 7	Palindromic Substrings	Interval	$dp[i][j]$ = is s[i..j] palindrome	$dp[i][i]=true$	ends-match && inner
51 6	Longest Palindromic Subseq	Interval	$dp[i][j]$ = LPS length in s[i..j]	$dp[i][i]=1$	ends-match: +2, else max
31 2	Burst Balloons	Interval	$dp[i][j]$ = max coins in open interval (i,j)	0	max over k: $dp[i][k]+coin+dp[k][j]$
62	Unique Paths	Grid	$dp[i][j]$ = paths to reach (i,j)	first row/col=1	$dp[i-1][j]+dp[i][j-1]$
63	Unique Paths II	Grid	$dp[i][j]$ = paths to reach (i,j) with obstacles	first row/col=1 until obstacle	$dp[i-1][j]+dp[i][j-1]$ if no obstacle, else 0
64	Min Path Sum	Grid	$dp[i][j]$ = min cost to reach (i,j)	first row/col cumulative	$\min(up, left)+grid[i][j]$
22 1	Maximal Square	Grid	$dp[i][j]$ = side of largest all-1 square at (i,j)	0	$\min(up, left, diag)+1$
27 9	Perfect Squares	Knapsack Unbounded	$dp[i]$ = min squares to sum to i	$dp[0]=0$, rest=INF	$\min(dp[i], dp[i-ij]+1)$ for $j \leq i$
32 9	Longest Increasing Path	Grid + Memo	memo[i][j] = LIP starting at (i,j)	0	$\max(dfs(neighbor))+1$
36 8	Largest Divisible Subset	Look-back 1D (LIS-style)	$dp[i]$ = max subset ending at $nums[i]$	$dp[i]=1$	$\max(dp[j]+1)$ if $nums[i] \% nums[j]==0$
12 1	Best Time I	State Machine	hold/cash, at most 1 buy	hold=-p[0]	hold=max(hold, -p[i])
12 2	Best Time II	State Machine	hold/cash, unlimited	hold=-p[0]	hold=max(hold, cash-p[i])

#	Name	Category	dp state	Init	Key transition
123	Best Time III	State Machine	buy1/sell1/buy2/sell2	buy1=buy2=-p[0]	chained 4 states
309	Stock with Cooldown	State Machine	hold/cash/cooldown	hold=-p[0]	cooldown=prevCash
714	Stock with Fee	State Machine	hold/cash	hold=-p[0]	sell: hold+p[i]-fee
746	Min Cost Climbing Stairs	Linear 1D	dp[i] = min cost to reach step i	dp[0]=dp[1]=0	min(dp[i-1]+cost[i-1], dp[i-2]+cost[i-2])
931	Minimum Falling Path Sum	Grid DP	dp[i][j] = min falling path sum to (i,j)	dp[0]=matrix[0]	matrix[i][j] + min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])
1049	Last Stone Weight II	Min possible weight of last stone after smashing	Partition into two near-equal subsets; 0/1 knapsack on sum/2	Reduce to subset sum: minimize S1 - S2 = total - 2*maxSubset≤sum/2	O(n*sum)
1312	Minimum Insertion Steps to Make Palindrome	Min insertions to make string palindrome	Interval DP: dp[i][j] = min insertions for s[i..j]; reduce to n - LPS	LPS reduction: answer = n - longestPalindromicSubsequence	O(n ²)

All Six Templates

```
// 1. LINEAR 1D – each cell depends on a fixed number of previous cells
// MENTAL MODEL: the answer at i is a fixed recipe over the last one or two answers.
// WHEN: "ways/cost to reach step i", "can't pick adjacent"
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++)
    dp[i] = f(dp[i-1], dp[i-2], ...);

// 2A. LOOK-BACK 1D – dp[i] depends on all j < i
// MENTAL MODEL: to finish position i, scan every earlier position j and extend the best valid one.
// WHEN: "longest increasing/chain ending here", "can the prefix be segmented"
int[] dp = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        dp[i] = f(dp[i], dp[j]); // e.g., LIS, word break
    }
}

// -----
// MENTAL MODEL: both are the same 2D recurrence collapsed to 1D.
// dp[i][j] = ways to make sum j using the first i items.
// After dropping the i dimension, dp[j-nums[i]] is read from:
// • the PREVIOUS row (item i not yet used) → 0/1, no reuse
// • the CURRENT row (item i already used) → unbounded, reuse
// Loop direction decides which one you read.
// Mnemonic: DESCENDING = Distinct (each once), ASCENDING = Again (reusable).
// Seed/operator picks the flavor:
// count → dp[0]=1, dp[j] += dp[j-w]
// max-value → dp[0]=0, dp[j] = Math.max(dp[j], dp[j-w] + v)
// -----

// 2B. KNAPSACK 0/1 – each item at most once
int[] dp = new int[target + 1];
dp[0] = 1; // 1 way to make 0: take nothing
for (int i = 0; i < nums.length; i++)
    for (int j = target; j >= nums[i]; j--) // ↓ DESCENDING
        dp[j] += dp[j - nums[i]];
// ^^^^^^^^^^^^^ j-nums[i] < j, not yet touched THIS pass
// → still "previous row" → item i unused → no reuse

// 2C. KNAPSACK UNBOUNDED – each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++)
    for (int j = nums[i]; j <= target; j++) // ↑ ASCENDING
        dp[j] += dp[j - nums[i]];
// ^^^^^^^^^^^^^ j-nums[i] < j, ALREADY updated THIS pass
// → "current row" → item i can reappear → reuse

// 3. 2D SEQUENCE – two strings/arrays; i indexes one, j indexes the other
// MENTAL MODEL: compare the last char of each prefix – match consumes both, mismatch drops one side.
// WHEN: "compare two strings/arrays" (LCS, edit distance, subsequence count)
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (match(i, j)) {
            dp[i][j] = dp[i-1][j-1] + val;
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1]);
        }
    }
}
}
```

```
// 4. INTERVAL DP – i goes RIGHT to LEFT; j goes i+1 to end
// "shorter comes first": when computing dp[i][j], dp[i+1][*] is already done
// MENTAL MODEL: build answers for short ranges first, then combine them into longer ranges.
// WHEN: "palindrome substring/subseq", "merge/burst in a range", split-point problems
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base; // length-1 intervals
for (int i = n - 1; i >= 0; i--) { // ← right-to-left
    for (int j = i + 1; j < n; j++) {
        dp[i][j] = f(dp[i+1][j-1], dp[i+1][j], dp[i][j-1]);
    }
}

// 5. GRID DP – 4-directional neighbors
// MENTAL MODEL: each cell's answer accumulates from the cells you could have arrived from.
// WHEN: "paths / min-cost in a grid moving right+down", "largest square"
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1}; // DOWN UP RIGHT LEFT
// Standard grid (dag, only right/down):
int[][] dp = new int[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dp[i][j] = grid[i][j] + combine(dp[i-1][j], dp[i][j-1]);

// 6. STATE MACHINE – named states updated simultaneously each step
// MENTAL MODEL: track the best value in each named situation; each day every state updates from yesterday's states.
// WHEN: "buy/sell/hold", "limited transactions", "cooldown/fee"
int cash = 0, hold = -prices[0];
for (int i = 1; i < prices.length; i++) {
    cash = Math.max(cash, hold + prices[i]);
    hold = Math.max(hold, ??? - prices[i]); // ??? depends on problem
}
```

Knapsack Decision Tree

```
Want count or min/max?
├─ Count (dp[j] += dp[j - num])
│   ├── 0/1 (each item once):    j descending → #416, #494
│   └─ Unbounded (reuse):       j ascending  → #518
│       └─ Ordered (permut):    target outer, nums inner → #377
└─ Min/Max (dp[j] = min/max(...))
    ├── 0/1 minimize:           j descending → #474 (maximize)
    └─ Unbounded minimize:     j ascending  → #322
```

Part 1 — Linear 1D DP

#70 Climbing Stairs

Description: Each step you can climb 1 or 2 steps. How many distinct ways to reach step n ?

Intuition: the last move was a 1-step or a 2-step, so ways to reach n = ways to $n-1$ + ways to $n-2$.

```
class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int[] dp = new int[n + 1];
        dp[1] = 1; dp[2] = 2;
        for (int i = 3; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
        return dp[n];
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#198 House Robber

Description: Rob houses on a line — can't rob two adjacent. Maximize total.

Intuition: at each house, either skip it (keep $dp[i-1]$) or rob it ($dp[i-2] + \text{value}$, since $i-1$ is now off-limits).

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        int[] dp = new int[n];
        dp[0] = nums[0]; dp[1] = Math.max(nums[0], nums[1]);
        for (int i = 2; i < n; i++)
            dp[i] = Math.max(dp[i-1], dp[i-2] + nums[i]);
        return dp[n-1];
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#213 House Robber II

Description: Houses arranged in a circle — first and last are adjacent. Can't rob both.

Key: run House Robber on $[0, n-2]$ and $[1, n-1]$; take the max. Two passes, exact same helper.

Intuition: breaking the circle means either the first house is excluded or the last is — solve both lines and keep the better.

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        return Math.max(rob(nums, 0, n - 2), rob(nums, 1, n - 1));
    }
    private int rob(int[] nums, int i, int j) {
        int prev2 = 0, prev1 = 0;
        for (int k = i; k <= j; k++) {
            int current = Math.max(prev1, prev2 + nums[k]);
            prev2 = prev1;
            prev1 = current;
        }
        return prev1;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#300 Longest Increasing Subsequence

Description: Length of the longest strictly increasing subsequence.

Template: Look-back 1D — $dp[i]$ depends on all $dp[j]$ where $j < i$ and $nums[j] < nums[i]$.

Intuition: the best subsequence ending at i extends the longest earlier one whose last value is still smaller.

```

class Solution {
    public int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        int max = 1;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
            }
            max = Math.max(max, dp[i]);
        }
        return max;
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#139 Word Break

Description: Can `s` be segmented into a sequence of dictionary words?

Template: Look-back 1D — `dp[i]` is true if some `dp[j]` is true and `s[j..i)` is a word.

Intuition: a prefix is breakable if some earlier breakable cut leaves a dictionary word as the final piece.

```

class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        Set<String> words = new HashSet<>(wordDict);
        int n = s.length();
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;
        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && words.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[n];
    }
}

```

Time $O(n^2 \cdot L)$ where L = substring length | **Space** $O(n)$

198 vs 300 vs 139 — Look-back patterns

	#198 House Robber	#300 LIS	#139 Word Break
dp[i] means:	max rob ending at i	LIS ending at i	can break s[0..i)
j range:	j = i-1, i-2 (fixed)	j = 0..i-1 (all)	j = 0..i-1 (all)
condition on j:	none (always)	nums[j] < nums[i]	dp[j] && word in dict
update:	max(dp[i-1], dp[i-2]+x)	max(dp[i], dp[j]+1)	dp[i] = true; break
answer:	dp[n-1]	max(dp)	dp[n]

Part 2 — Knapsack DP

Canonical Templates

```
// — 0/1 KNAPSACK — each item at most once
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = target; j >= nums[i]; j--) // DESCENDING: sees OLD dp (before item i)
        dp[j] += dp[j - nums[i]];
}

// — UNBOUNDED KNAPSACK — each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = nums[i]; j <= target; j++) // ASCENDING: sees NEW dp (after item i already used)
        dp[j] += dp[j - nums[i]];
}

// — PERMUTATION COUNT — order matters (Combination Sum IV)
int[] dp = new int[target + 1];
dp[0] = 1;
for (int j = 1; j <= target; j++) { // OUTER: target
    for (int num : nums) { // INNER: try all nums (not fixing order)
        if (j >= num) dp[j] += dp[j - num];
    }
}
```

Why descending for 0/1:

`dp[j - nums[i]]` is read **before** `dp[j]` is updated ($j > j - \text{nums}[i]$). So it always sees the dp state from before item `i` was considered — no item is used twice.

Why ascending for unbounded:

`dp[j - nums[i]]` was already updated in this same pass ($j - \text{nums}[i] < j$, processed earlier). This allows item `i` to be picked multiple times.

2D anchor: both loops are `dp[i][j] = dp[i-1][j] + dp[i-?][j-w]` collapsed to 1D — descending keeps `dp[j-w]` on the *previous* row (item `i` unused), ascending pulls it from the *current* row (item `i` reused).

Mnemonic: DESCENDING = **D**istinct (each item once), ASCENDING = **A**gain (reusable).

Flavor = seed + operator (same skeleton):

- count → `dp[0]=1` , `dp[j] += dp[j-w]`
- feasibility → `dp[0]=true` , `dp[j] = dp[j] || dp[j-w]`
- max-value → `dp[0]=0` , `dp[j] = Math.max(dp[j], dp[j-w] + v)`

#416 Partition Equal Subset Sum

Description: Can the array be partitioned into two subsets with equal sum?

Key: equivalent to: can we pick a subset summing to `total/2` ?


```

class Solution {
    public boolean canPartition(int[] nums) {
        int sum = Arrays.stream(nums).sum();
        if (sum % 2 != 0) return false;
        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int num : nums) {
            for (int j = target; j >= num; j--)    // 0/1: descending
                dp[j] = dp[j] || dp[j - num];
        }
        return dp[target];
    }
}

```

#494 Target Sum

Description: Assign `+` or `-` to each number to reach `target`. How many ways?

Key: assign `+` to subset P and `-` to subset N. Then $P - N = \text{target}$ and $P + N = \text{sum}$. So $P = (\text{sum} + \text{target}) / 2$. Count subsets summing to P.

```

class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = Arrays.stream(nums).sum();
        if ((sum + target) % 2 != 0 || Math.abs(target) > sum) return 0;
        int t = (sum + target) / 2;
        int[] dp = new int[t + 1];
        dp[0] = 1;
        for (int num : nums) {
            for (int j = t; j >= num; j--)    // 0/1: descending
                dp[j] += dp[j - num];
        }
        return dp[t];
    }
}

```

#474 Ones and Zeroes

Description: Given strings of '0'/'1', find the largest subset using at most `m` zeros and `n` ones.

Key: 2D 0/1 knapsack — two capacity dimensions. Both must iterate descending.

```

class Solution {
    public int findMaxForm(String[] strs, int m, int n) {
        int[][] dp = new int[m + 1][n + 1];
        for (String s : strs) {
            int zeros = 0, ones = 0;
            for (char c : s.toCharArray()) { if (c == '0') zeros++; else ones++; }
            for (int i = m; i >= zeros; i--)    // 0/1: descending (both dims)
                for (int j = n; j >= ones; j--)
                    dp[i][j] = Math.max(dp[i][j], dp[i - zeros][j - ones] + 1);
        }
        return dp[m][n];
    }
}

```

#322 Coin Change

Description: Minimum number of coins to make `amount`. Coins can be reused.

Key: unbounded knapsack (ascending), but minimize instead of count — seed with INF, take min.

```

class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);          // impossible sentinel
        dp[0] = 0;
        for (int coin : coins) {
            for (int j = coin; j <= amount; j++) // unbounded: ascending
                dp[j] = Math.min(dp[j], dp[j - coin] + 1);
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}

```

#518 Coin Change II

Description: Number of distinct combinations (order doesn't matter) to make `amount` .

Key: unbounded knapsack (ascending), count variant — combinations, not permutations.

```

class Solution {
    public int change(int amount, int[] coins) {
        int[] dp = new int[amount + 1];
        dp[0] = 1;
        for (int coin : coins) {
            for (int j = coin; j <= amount; j++) // unbounded: ascending
                dp[j] += dp[j - coin];
        }
        return dp[amount];
    }
}

```

#377 Combination Sum IV

Description: Number of ordered sequences (order matters) that sum to `target` .

Key: permutation count — swap loop order: target outer, nums inner. This tries all nums for each partial sum, so different orderings are counted separately.

```

class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target + 1];
        dp[0] = 1;
        for (int j = 1; j <= target; j++) {      // outer: target value
            for (int num : nums) {              // inner: all numbers
                if (j >= num) dp[j] += dp[j - num];
            }
        }
        return dp[target];
    }
}

```

518 vs 377 — Side by Side

	#518 Coin Change II	#377 Combination Sum IV
Loop order:	coins outer, j inner	j outer, nums inner
Counts:	combinations (sets)	permutations (sequences)
Example (1,2→3):	{1,2} and {1,1,1} = 2	{1,2},{2,1},{1,1,1} = 3
Why:	fixing coin order = no dup	re-trying all nums each j = all orders

Part 3 — 2D Sequence DP

Canonical Template

```
// i indexes string/array 1 (1-indexed), j indexes string/array 2
int[][] dp = new int[m + 1][n + 1];
// Init dp[0][*] and dp[*][0] based on problem
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i-1) == s2.charAt(j-1)) {
            dp[i][j] = dp[i-1][j-1] + 1;           // characters match
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
        }
    }
}
```

#1143 Longest Common Subsequence

Description: Length of the longest subsequence present in both strings.

Intuition: if the last chars match, they're part of the LCS (+1 on the diagonal); otherwise drop one char from whichever string and keep the better.

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i-1) == text2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }
        return dp[m][n];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#72 Edit Distance

Description: Minimum operations (insert, delete, replace) to convert `word1` to `word2`.

Init: $dp[i][0] = i$ (delete all), $dp[0][j] = j$ (insert all).

Intuition: matched chars cost nothing (diagonal); otherwise take the cheapest of replace/delete/insert and add 1.

```

class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1]; // match: no cost
                } else {
                    dp[i][j] = 1 + Math.min(dp[i-1][j-1], // replace
                                           Math.min(dp[i-1][j], // delete
                                                    dp[i][j-1])); // insert
                }
            }
        }
        return dp[m][n];
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#115 Distinct Subsequences

Description: Number of ways `s` contains `t` as a subsequence.

Init: `dp[i][0] = 1` (empty `t` matched by any `s` prefix).

Intuition: you can always skip the current char of `s`; if it matches the current char of `t`, you may also use it to advance `t`.

```

class Solution {
    public int numDistinct(String s, String t) {
        int m = s.length(), n = t.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = 1;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                dp[i][j] = dp[i-1][j]; // always: skip s[i]
                if (s.charAt(i-1) == t.charAt(j-1))
                    dp[i][j] += dp[i-1][j-1]; // also: use s[i] to match t[j]
            }
        }
        return dp[m][n];
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

1143 vs 72 vs 115 — Transition Cheat Sheet

	#1143 LCS	#72 Edit Distance	#115 Distinct Subseq
Match (<code>s[i]==t[j]</code>):	<code>dp[i-1][j-1]+1</code>	<code>dp[i-1][j-1]</code>	<code>dp[i-1][j]+dp[i-1][j-1]</code>
No match:	<code>max(up, left)</code>	<code>1+min(diag, up, left)</code>	<code>dp[i-1][j]</code> (skip <code>s[i]</code>)
Init <code>dp[0][*]</code> :	0	<code>j</code> (insert <code>j</code> chars)	0
Init <code>dp[*][0]</code> :	0	<code>i</code> (delete <code>i</code> chars)	1 (empty <code>t</code> always matches)

Part 4 — Interval DP

Canonical Template

Rule: `i` goes right-to-left (ensures shorter/inner intervals are computed first). `j` goes left-to-right from `i+1`. When computing `dp[i][j]`, all `dp[i'][j']` with `i' > i` or `j' < j` are already done.

```
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base_case; // single elements
for (int i = n - 1; i >= 0; i--) { // right-to-left
    for (int j = i + 1; j < n; j++) { // left-to-right from i+1
        // dp[i][j] can safely use dp[i+1][j], dp[i][j-1], dp[i+1][j-1]
        dp[i][j] = ...;
    }
}
```

Alternative (length-based): Equivalent, sometimes clearer for k-partition problems:

```
for (int len = 2; len <= n; len++) { // outer: length of interval
    for (int i = 0; i + len - 1 < n; i++) { // inner: left endpoint
        int j = i + len - 1;
        dp[i][j] = ...;
    }
}
```

#647 Palindromic Substrings

Description: Count all palindromic substrings of `s`.

Intuition: `s[i..j]` is a palindrome when its two ends match AND the inside `s[i+1..j-1]` already is.

```
class Solution {
    public int countSubstrings(String s) {
        int n = s.length(), count = 0;
        boolean[][] dp = new boolean[n][n];
        for (int i = n - 1; i >= 0; i--) {
            for (int j = i; j < n; j++) {
                dp[i][j] = s.charAt(i) == s.charAt(j)
                    && (j - i < 2 || dp[i+1][j-1]); // ends match && inner is palindrome
                if (dp[i][j]) count++;
            }
        }
        return count;
    }
}
```

Time $O(n^2)$ | **Space** $O(n^2)$

#516 Longest Palindromic Subsequence

Description: Length of the longest subsequence of `s` that is a palindrome.

Intuition: matching ends add 2 to the inner range's best; otherwise drop one end and keep the larger.

```

class Solution {
    public int longestPalindromeSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) dp[i][i] = 1;
        for (int i = n - 1; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return dp[0][n-1];
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#312 Burst Balloons

Description: Burst all balloons to maximize coins. Coins from bursting balloon `k` between open boundaries `i` and `j` = `balloons[i] * balloons[k] * balloons[j]`.

Key insight: think of `k` as the **last** balloon burst in range `(i, j)` (open interval). Adding sentinel `1` s at both ends simplifies boundary cases.

Template: length-based outer loop; inner loop over last-burst position `k`.

```

class Solution {
    public int maxCoins(int[] nums) {
        int n = nums.length;
        int[] b = new int[n + 2];
        b[0] = b[n + 1] = 1;
        for (int i = 0; i < n; i++) b[i + 1] = nums[i];
        int size = n + 2;
        int[][] dp = new int[size][size];
        for (int len = 2; len < size; len++) {
            for (int i = 0; i + len < size; i++) {
                int j = i + len;
                for (int k = i + 1; k < j; k++) { // k = last balloon burst in (i, j)
                    dp[i][j] = Math.max(dp[i][j],
                        dp[i][k] + b[i] * b[k] * b[j] + dp[k][j]);
                }
            }
        }
        return dp[0][n + 1];
    }
}

```

Time $O(n^3)$ | **Space** $O(n^2)$

Part 5 — Grid DP

Canonical Template

```
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};    // DOWN UP RIGHT LEFT

// Standard grid (only move right/down - DAG, no cycle):
int[][] dp = new int[rows][cols];
// initialize first row and first column
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dp[i][j] = grid[i][j] + f(dp[i-1][j], dp[i][j-1]);

// All-direction grid (memoized DFS for increasing/decreasing paths):
int[][] memo = new int[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dfs(grid, i, j, memo, dr, dc);
```

#62 Unique Paths

Description: Number of paths from top-left to bottom-right, moving only right or down.

Init: first row and column are all 1 (only one way to reach any edge cell).

Intuition: you reach a cell only from above or from the left, so its path count is the sum of those two.

```
class Solution {
    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) dp[i][0] = 1;
        for (int j = 0; j < n; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#64 Minimum Path Sum

Description: Minimum sum path from top-left to bottom-right, moving only right or down.

Init: first row/column are cumulative sums (no choice on edges).

Intuition: the cheapest way into a cell is its own value plus the cheaper of the cell above or to its left.

```
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int[][] dp = new int[m][n];
        dp[0][0] = grid[0][0];
        for (int i = 1; i < m; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
        for (int j = 1; j < n; j++) dp[0][j] = dp[0][j-1] + grid[0][j];
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1]);
        return dp[m-1][n-1];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#221 Maximal Square

Description: Find the largest all-1 square in a binary matrix. Return its area.

Key insight: $dp[i][j]$ = side length of the largest all-1 square with its bottom-right corner at (i, j) . Limited by the minimum of up, left, and diagonal neighbors.

```
class Solution {
    public int maximalSquare(char[][] matrix) {
        int m = matrix.length, n = matrix[0].length, maxSide = 0;
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == '1') {
                    dp[i][j] = (i == 0 || j == 0) ? 1
                        : Math.min(dp[i-1][j], Math.min(dp[i][j-1], dp[i-1][j-1])) + 1;
                    maxSide = Math.max(maxSide, dp[i][j]);
                }
            }
        }
        return maxSide * maxSide;
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#329 Longest Increasing Path in a Matrix

Description: Find the longest strictly increasing path. Can move in all 4 directions.

Template: DFS + memoization (grid has ordering from values, so no cycle in the recursion DAG).

Intuition: strictly increasing values forbid revisiting, so each cell's longest path is 1 + the best of its larger neighbors — cache it.

```
class Solution {
    int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};

    public int longestIncreasingPath(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        int[][] memo = new int[m][n];
        int max = 0;
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                max = Math.max(max, dfs(matrix, i, j, memo));
        return max;
    }
    private int dfs(int[][] matrix, int i, int j, int[][] memo) {
        if (memo[i][j] != 0) return memo[i][j];
        memo[i][j] = 1;
        for (int dir = 0; dir < 4; dir++) {
            int ni = i + dr[dir], nj = j + dc[dir];
            if (ni >= 0 && ni < matrix.length && nj >= 0 && nj < matrix[0].length
                && matrix[ni][nj] > matrix[i][j])
                memo[i][j] = Math.max(memo[i][j], dfs(matrix, ni, nj, memo) + 1);
        }
        return memo[i][j];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

Part 6 — State Machine DP (Stock Problems)

Canonical States

```
cash      = max profit when NOT holding (free to buy or idle)
hold      = max profit when HOLDING stock (bought it at some cost)
cooldown = max profit right after selling (can't buy today)
```

All Five Transitions

	buy	sell	re-buy condition
#121 (1 tx):	hold = max(hold, -price)		no re-buy (ignore cash)
#122 (unlimited):	hold = max(hold, cash - price)		re-buy with full cash
#123 (2 tx):	chain: buy2 uses sell1 cash		4 states chained
#309 (cooldown):	hold = max(hold, cooldown - price)		must wait 1 day
#714 (fee):	hold = max(hold, cash - price)		pay fee on sell

#121 Best Time to Buy and Sell Stock (at most 1 transaction)

Intuition: `hold` is the best profit if you currently own a share bought at the lowest price seen; sell whenever today's price beats that.

```
class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, -prices[i]); // no cash reinvestment (1 buy)
        }
        return cash;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#122 Best Time to Buy and Sell Stock II (unlimited transactions)

Key difference from #121: `hold = max(hold, cash - prices[i])` — can reinvest all prior profits.

Intuition: with unlimited trades, buying can fund itself from accumulated `cash`, so capture every upward move.

```
class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, cash - prices[i]); // reinvest cash (unlimited buys)
        }
        return cash;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#123 Best Time to Buy and Sell Stock III (at most 2 transactions)

Key: 4 named states chained in order: `buy1` → `sell1` → `buy2` → `sell2`.

Intuition: the second buy can only spend profit left after the first sell, so chain the states in trade order.

```

class Solution {
    public int maxProfit(int[] prices) {
        int buy1 = -prices[0], sell1 = 0, buy2 = -prices[0], sell2 = 0;
        for (int i = 1; i < prices.length; i++) {
            buy1 = Math.max(buy1, -prices[i]);
            sell1 = Math.max(sell1, buy1 + prices[i]);
            buy2 = Math.max(buy2, sell1 - prices[i]);
            sell2 = Math.max(sell2, buy2 + prices[i]);
        }
        return sell2;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#309 Best Time to Buy and Sell Stock with Cooldown

Description: After selling, must wait 1 day before buying again.

Key: `hold` can only use `cooldown` (yesterday's `cash`), not today's `cash`. Save `prevCash` before updating.

Intuition: the cooldown forces buying to draw from cash that's at least one day old, so carry yesterday's cash forward.

```

class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0], cooldown = 0;
        for (int i = 1; i < prices.length; i++) {
            int prevCash = cash;
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, cooldown - prices[i]); // buy only from cooldown state
            cooldown = prevCash; // yesterday's cash
        }
        return cash;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#714 Best Time to Buy and Sell Stock with Transaction Fee

Description: Pay a fee per transaction (on sell). Unlimited transactions.

Key: same as #122 but subtract `fee` when selling.

Intuition: the fee just shrinks every sale, so a trade is only worth making when the gain clears the fee.

```

class Solution {
    public int maxProfit(int[] prices, int fee) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i] - fee); // pay fee on sell
            hold = Math.max(hold, cash - prices[i]);
        }
        return cash;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Stock Problems — Differences at a Glance

Problem	hold update	cash update	Extra state
#121	<code>max(hold, -price)</code>	<code>max(cash, hold+price)</code>	none
#122	<code>max(hold, cash-price)</code>	<code>max(cash, hold+price)</code>	none
#123	4 variables, chained	-	<code>sell1</code> feeds <code>buy2</code>
#309	<code>max(hold, cooldown-price)</code>	<code>max(cash, hold+price)</code>	<code>cooldown = prevCash</code>
#714	<code>max(hold, cash-price)</code>	<code>max(cash, hold+price-fee)</code>	none

Template Chooser

Signal in problem	Template
<code>dp[i]</code> depends on <code>dp[i-1]</code> , <code>dp[i-2]</code>	Linear 1D — fixed lookback
<code>dp[i]</code> depends on all <code>dp[j]</code> where <code>j < i</code>	Look-back 1D — $O(n^2)$ inner loop
Each item used at most once, sum target	Knapsack 0/1 — <code>j</code> descending
Items reusable, sum target, order irrelevant	Knapsack Unbounded — <code>j</code> ascending
Items reusable, order matters (permutations)	Permutation count — target outer
Two strings/arrays, compare characters	2D Sequence DP
Palindrome, substring merge, balloon burst	Interval DP — <code>i</code> right-to-left
Grid, only right/down movement	Grid DP — cumulative from edges
Grid, all 4 directions, monotone constraint	DFS + memo (no cycle in DAG)
Buy/sell/hold decision changes day to day	State Machine — named states

#746 Min Cost Climbing Stairs

Description: Each step has a cost. You can start from index 0 or 1, and from each step you can climb 1 or 2 steps. Return the minimum cost to reach the top (past the last index).

Algorithm: 1D DP. `dp[i]` = min cost to reach step `i`. `dp[0] = dp[1] = 0` (can start for free). For `i >= 2`: `dp[i] = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2])`.

```
class Solution {
    public int minCostClimbingStairs(int[] cost) {
        int n = cost.length;
        int[] dp = new int[n + 1];
        for (int i = 2; i <= n; i++)
            dp[i] = Math.min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2]); // ← two sources
        return dp[n];
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#91 Decode Ways

Description: A string of digits can be decoded: '1'-'9' → single letter, "10"-"26" → two-digit letter. Count total decoding ways.

Variation: Two sources at each position. If the current digit is non-zero, it can be decoded alone (`dp[i] += dp[i-1]`). If the previous two digits form a valid two-letter code (10-26), add `dp[i-2]`.

```

class Solution {
    public int numDecodings(String s) {
        if (s.charAt(0) == '0') return 0;
        int n = s.length();
        int[] dp = new int[n + 1];
        dp[0] = 1; dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            int oneDigit = s.charAt(i-1) - '0';
            int twoDigit = Integer.parseInt(s.substring(i-2, i));
            if (oneDigit != 0) dp[i] += dp[i-1]; // ← VARIATION: single digit (1-9)
            if (twoDigit >= 10 && twoDigit <= 26) dp[i] += dp[i-2]; // ← VARIATION: two digits (10-26)
        }
        return dp[n];
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#279 Perfect Squares

Description: Return the minimum number of perfect squares (1, 4, 9, 16, ...) that sum to n .

Algorithm: Unbounded knapsack. $dp[i]$ = min squares to reach sum i . For each i , try all squares $j*j \leq i$: $dp[i] = \min(dp[i], dp[i - j*j] + 1)$.

```

class Solution {
    public int numSquares(int n) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n + 1);
        dp[0] = 0;
        for (int i = 1; i <= n; i++)
            for (int j = 1; j * j <= i; j++) // ← VARIATION: iterate all squares ≤ i
                dp[i] = Math.min(dp[i], dp[i - j*j] + 1);
        return dp[n];
    }
}

```

Time $O(n\sqrt{n})$ | **Space** $O(n)$

#96 Unique Binary Search Trees

Description: Return the number of structurally unique BSTs that store values 1 to n .

Variation: Catalan number recurrence. $dp[i]$ = number of unique BSTs with i nodes. When root = j , left subtree has $j-1$ nodes, right subtree has $i-j$ nodes: $dp[i] += dp[j-1] * dp[i-j]$.

```

class Solution {
    public int numTrees(int n) {
        int[] dp = new int[n + 1];
        dp[0] = dp[1] = 1;
        for (int i = 2; i <= n; i++)
            for (int j = 1; j <= i; j++)
                dp[i] += dp[j-1] * dp[i-j]; // ← VARIATION: Catalan: root=j, left=j-1, right=i-j
        return dp[n];
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#63 Unique Paths II

Description: A robot moves from top-left to bottom-right in an $m \times n$ grid with obstacles. Count unique paths. Cells with `obstacleGrid[i][j] == 1` are blocked.

Variation: Same as #62 (Unique Paths) but skip cells with obstacles: `dp[i][j] = 0` if blocked, else `dp[i-1][j] + dp[i][j-1]`.

```
class Solution {
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
        int m = obstacleGrid.length, n = obstacleGrid[0].length;
        int[][] dp = new int[m][n];
        for (int i = 0; i < m && obstacleGrid[i][0] == 0; i++) dp[i][0] = 1; // ← VARIATION: stop at obstacle
        for (int j = 0; j < n && obstacleGrid[0][j] == 0; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                if (obstacleGrid[i][j] == 0) // ← VARIATION: skip blocked cells
                    dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#583 Delete Operation for Two Strings

Description: Return the minimum number of deletions to make `word1` and `word2` equal.

Variation: Reduce to LCS. Minimum deletions = `len(word1) + len(word2) - 2 * LCS(word1, word2)`. Compute LCS using standard 2D DP.

```
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m+1][n+1]; // dp[i][j] = LCS length
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }
        return m + n - 2 * dp[m][n]; // ← VARIATION: reduce to LCS
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#368 Largest Divisible Subset

Description: Find the largest subset of distinct positive integers such that for every pair (a, b) , either $a \% b == 0$ or $b \% a == 0$.

Variation: Sort first, then apply LIS-style DP. `dp[i]` = size of largest divisible subset ending at `nums[i]`. If `nums[i] % nums[j] == 0`, then `dp[i] = max(dp[i], dp[j] + 1)`. Track parent indices to reconstruct the subset.

```

class Solution {
    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums); // ← VARIATION: sort required for divisibility chaining
        int n = nums.length;
        int[] dp = new int[n], parent = new int[n];
        Arrays.fill(dp, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
        int maxLen = 1, maxIdx = 0;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] % nums[j] == 0 && dp[j] + 1 > dp[i]) { // ← VARIATION: divisibility check
                    dp[i] = dp[j] + 1;
                    parent[i] = j;
                }
            }
            if (dp[i] > maxLen) { maxLen = dp[i]; maxIdx = i; }
        }
        List<Integer> result = new ArrayList<>();
        int i = maxIdx;
        while (parent[i] != i) { result.add(nums[i]); i = parent[i]; }
        result.add(nums[i]);
        Collections.reverse(result);
        return result;
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#931 Minimum Falling Path Sum

Description: Given an $n \times n$ matrix, return the minimum sum of a falling path. A falling path starts at any element in the first row and chooses the element directly below or diagonally adjacent in each row.

Variation: Grid DP with three sources from the row above. $dp[i][j] = matrix[i][j] + \min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])$.

```

class Solution {
    public int minFallingPathSum(int[][] matrix) {
        int n = matrix.length;
        int[][] dp = new int[n][n];
        dp[0] = matrix[0].clone();
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int best = dp[i-1][j];
                if (j > 0) best = Math.min(best, dp[i-1][j-1]); // ← VARIATION: three sources
                if (j < n-1) best = Math.min(best, dp[i-1][j+1]);
                dp[i][j] = matrix[i][j] + best;
            }
        }
        int min = Integer.MAX_VALUE;
        for (int val : dp[n-1]) min = Math.min(min, val);
        return min;
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#10 Regular Expression Matching

Description: Implement regex matching with `.` (any single char) and `*` (zero or more of the preceding element). Match must cover the entire input string. **Variation:** 2D DP. $dp[i][j]$ = whether $s[0..i)$ matches $p[0..j)$. The `*` has two cases: zero occurrences ($dp[i][j-2]$) or one-more occurrence when the preceding pattern char matches $s[i-1]$.

```

class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m+1][n+1];
        dp[0][0] = true;
        for (int j = 1; j <= n; j++)
            if (p.charAt(j-1) == '*') dp[0][j] = dp[0][j-2]; // ← VARIATION: '*' can erase preceding
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (p.charAt(j-1) == '*') {
                    dp[i][j] = dp[i][j-2]; // ← VARIATION: zero occurrence
                    if (matches(s, p, i, j-1))
                        dp[i][j] = dp[i][j] || dp[i-1][j]; // ← VARIATION: one more occurrence
                } else if (matches(s, p, i, j)) {
                    dp[i][j] = dp[i-1][j-1];
                }
            }
        }
        return dp[m][n];
    }
    private boolean matches(String s, String p, int i, int j) {
        return p.charAt(j-1) == '.' || s.charAt(i-1) == p.charAt(j-1);
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#87 Scramble String

Description: Given s_1 and s_2 , return true if s_2 is a scrambled string of s_1 (formed by recursively partitioning into two non-empty substrings and optionally swapping them). **Variation:** memoized recursion over (i_1, i_2, len) . At each length, try every split point both swapped and non-swapped.

```

class Solution {
    private Map<String, Boolean> memo = new HashMap<>();
    public boolean isScramble(String s1, String s2) {
        if (s1.equals(s2)) return true;
        if (s1.length() != s2.length()) return false;
        String key = s1 + "#" + s2;
        if (memo.containsKey(key)) return memo.get(key);
        int n = s1.length();
        int[] count = new int[26];
        for (int i = 0; i < n; i++) { count[s1.charAt(i) - 'a']++; count[s2.charAt(i) - 'a']--; }
        for (int c : count) if (c != 0) { memo.put(key, false); return false; } // char mismatch prune
        for (int i = 1; i < n; i++) {
            // no swap
            if (isScramble(s1.substring(0, i), s2.substring(0, i))
                && isScramble(s1.substring(i), s2.substring(i))) { memo.put(key, true); return true; }
            // swap // ← VARIATION: try swapped partitions
            if (isScramble(s1.substring(0, i), s2.substring(n-i))
                && isScramble(s1.substring(i), s2.substring(0, n-i))) { memo.put(key, true); return true; }
        }
        memo.put(key, false); return false;
    }
}

```

Time $O(n^4)$ | **Space** $O(n^3)$

#1049 Last Stone Weight II

Description: Repeatedly smash the two heaviest stones (result is their difference). Return the smallest possible weight of the remaining stone.

Variation: equivalent to splitting stones into two groups to minimize the difference of their sums. 0/1 knapsack to find the max subset sum $\leq \text{total}/2$.

```

class Solution {
    public int lastStoneWeightII(int[] stones) {
        int total = 0;
        for (int s : stones) total += s;
        int target = total / 2;          // ← VARIATION: subset sum target
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int s : stones)
            for (int j = target; j >= s; j--)    // ← 0/1 knapsack: j descending
                dp[j] = dp[j] || dp[j - s];
        for (int j = target; j >= 0; j--)
            if (dp[j]) return total - 2 * j;    // ← VARIATION: minimize |S1-S2|
        return 0;
    }
}

```

Time $O(n \cdot \text{sum})$ | **Space** $O(\text{sum})$

#1312 Minimum Insertion Steps to Make a String Palindrome

Description: Return the minimum number of insertions needed to make a string a palindrome. **Variation:** answer = $n - \text{longest palindromic subsequence (LPS)}$. LPS = LCS of the string and its reverse; or solve directly via interval DP.

```

class Solution {
    public int minInsertions(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];          // dp[i][j] = LPS length in s[i..j]
        for (int i = n - 1; i >= 0; i--) {    // ← interval DP: shorter spans first
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return n - dp[0][n-1];              // ← VARIATION: insertions = n - LPS
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$